

This guide demonstrates how to create an animated main menu background in Hol4. The finished example can be found on the Steam Workshop link below.

This guide assumes that you are using Windows 10 and GIMP.

<https://steamcommunity.com/sharedfiles/filedetails/?id=2336744284>

Clipping the video

First of all, the source video has to be clipped to cut out the desired parts, and its frame rate lowered to about 24 fps for optimal size. Frame rate above 24 fps increases the amount of work and final file size significantly for little perceptual gain, while frame rate below that looks like a series of discrete images instead of an animation.

The tool this guide uses for video editing is [ffmpeg](#). ffmpeg is a command line tool, so you have to get comfortable with it. You can use another program for this purpose if you have a preference, but you'll have to find the equivalent operations yourself.

An ffmpeg command looks like this:

```
"D:\Programs\ffmpeg\bin\ffmpeg.exe" -i "D:\Youjo Senki\movie.mkv" -vf  
select='between(t,2057,2074)' -vsync 0 out%d.png
```

To break it down:

```
<path to ffmpeg.exe> -i <input file> <list of commands to run> <output file>
```

Double quotes around path/file names are only necessary if the path or file name contains spaces, but always including them reduces the chance of errors.

When specifying the output file's path, make sure the folders are already created! ffmpeg can create output files, but it doesn't create new folders if it can't find the folder specified.

Simple clipping

If you only want a single section of a video, then the following command will suffice:

```
<ffmpeg> -i <input> -c:v copy -vf select='between(t,<start time>,<end time>)'  
<output>
```

Replace the yellow parts with the appropriate paths and values. Times are measured in seconds and accept decimals. For example:

```
"D:\Programs\ffmpeg\bin\ffmpeg.exe" -i "D:\Youjo Senki\movie.mkv" -c:v copy -vf  
select='between(t,1490.5,1452.1)' "D:\Youjo Senki\edit\clip.mkv"
```

Complex clipping

The following command allows you to cut an arbitrary number of sections then concatenate them:

```
<ffmpeg> -i <input> -filter_complex  
"[0:v]trim=start=<start_time_1>:end=<end_time_1>,setpts=PTS-STARTPTS[a];  
[0:v]trim=start=<start_time_2>:end=<end_time_2>,setpts=PTS-STARTPTS[b];  
[a][b]concat[c];  
[0:v]trim=start=<start_time_3>:end=<end_time_3>,setpts=PTS-STARTPTS[d];  
[c][d]concat[out1]" -map [out1] <output>
```

Repeat the trim-concat parts for as many times you need. The “markers” ([a], [b], [out1], etc.) can be arbitrary letters or strings.

Changing the framerate

You can clip and change the frame rate in one command without creating intermediate files, but for simplicity’s sake we’ll do it in a separate command:

```
<ffmpeg> -i <input> -filter:v fps=fps=24 <output>
```

Somewhere near the bottom you should see a message like this. Write down the number of frames because we’ll need it later.

```
frame= 168 fps= 69 q=-1.0 Lsize=    1398kB time=00:00:06.88 bitrate=1664.0kbits/s  
speed=2.82x
```

If you accidentally closed the window before marking it down, use this command

```
<ffmpeg> -i <final video file> -map 0:v:0 -c copy -f null -
```

To make ffmpeg do a dry run and show the number of frames in a similar output message.

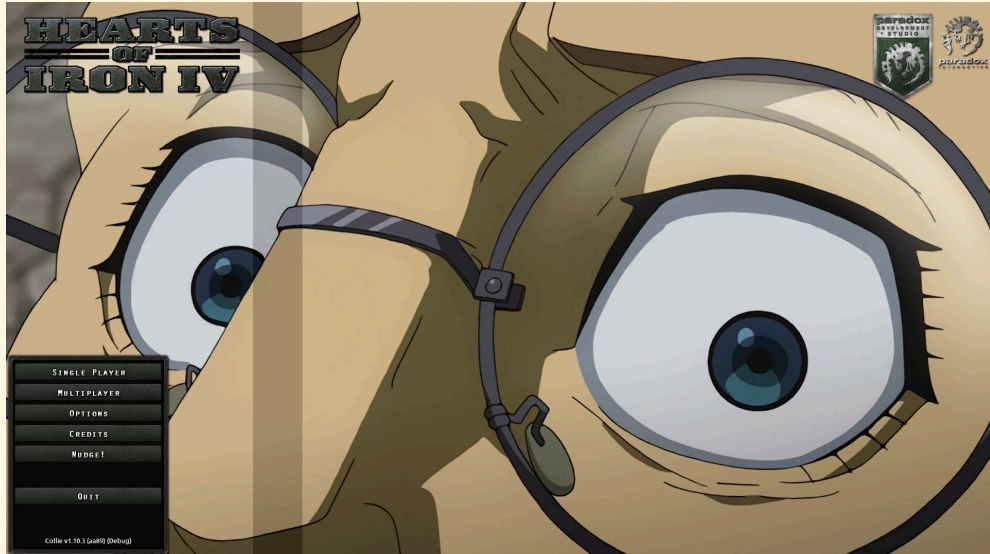
Do the math

The key constraints to playing videos in Hol4 (other than no native video support) are:

1. The texture handler can’t handle anything with a dimension of over 16384 pixels
2. frameAnimatedSpriteType only accepts single row sprite sheets

This means that for a single frameAnimatedSpriteType, [width]*[num of frames] must not exceed 16384. For a 1080p image (1920*1080), this means that there can only be $16384/1920 \approx 8$ frames.

These restrictions, however, can be bypassed by replacing the menu background with multiple images:



The highlighted (or lowlighted to be precise) area is one out of the 20 “strips” in the example.

The video is 1920 pixels wide, and we have 168 frames (from the section above), so we have $1920 \times 168 = 322560$ pixels. Each texture can only contain 16384 pixels in width, so we need at least $322560 / 16384 = 19.6875$ textures. We round that up to 20, so we’ll divide the loading screen into 20 pieces of texture. Now we have to calculate the width of each frame, which is $1920 / 20 = 96$ pixels. Note that if the width isn’t divisible by the number of textures in the last step, you might want to either divide the texture further to find a number divisible by the width, or give the last texture a different width than the rest.

Now that we have the numbers (20 textures, 96 pixels wide), we can start chopping the video into sliced frames.

Grabbing the frames from the video:

The following commands will export every frames in all 20 96 pixel wide strips into their respective folders:

```
<ffmpeg> -i <input> -vf crop=96:1080:0:0 <output>01\out%d.png
<ffmpeg> -i <input> -vf crop=96:1080:96:0 <output>02\out%d.png
<ffmpeg> -i <input> -vf crop=96:1080:192:0 <output>03\out%d.png
<ffmpeg> -i <input> -vf crop=96:1080:288:0 <output>04\out%d.png
<ffmpeg> -i <input> -vf crop=96:1080:384:0 <output>05\out%d.png
<ffmpeg> -i <input> -vf crop=96:1080:480:0 <output>06\out%d.png
<ffmpeg> -i <input> -vf crop=96:1080:576:0 <output>07\out%d.png
<ffmpeg> -i <input> -vf crop=96:1080:672:0 <output>08\out%d.png
<ffmpeg> -i <input> -vf crop=96:1080:768:0 <output>09\out%d.png
<ffmpeg> -i <input> -vf crop=96:1080:864:0 <output>10\out%d.png
<ffmpeg> -i <input> -vf crop=96:1080:960:0 <output>11\out%d.png
<ffmpeg> -i <input> -vf crop=96:1080:1056:0 <output>12\out%d.png
<ffmpeg> -i <input> -vf crop=96:1080:1152:0 <output>13\out%d.png
<ffmpeg> -i <input> -vf crop=96:1080:1248:0 <output>14\out%d.png
<ffmpeg> -i <input> -vf crop=96:1080:1344:0 <output>15\out%d.png
<ffmpeg> -i <input> -vf crop=96:1080:1440:0 <output>16\out%d.png
<ffmpeg> -i <input> -vf crop=96:1080:1536:0 <output>17\out%d.png
<ffmpeg> -i <input> -vf crop=96:1080:1632:0 <output>18\out%d.png
<ffmpeg> -i <input> -vf crop=96:1080:1728:0 <output>19\out%d.png
<ffmpeg> -i <input> -vf crop=96:1080:1824:0 <output>20\out%d.png
```

Explanations:

`crop=<width>:<height>:<horizontal offset>:<vertical offset>`

What we're doing here is cropping the input to 96*1080, and offset it further by 96 pixels for every iteration.

[%d] means the index number. Since we're turning the video into images, we have to give each frame a different name based on their index, so the output files will be named [out1.png], [out2.png], ... [out168.png].

As mentioned earlier, the output folders must exist as ffmpeg only creates files not folders. If you don't mind all of them being dumped into the same folder, you can do something like `<output folder>\20-%d.png` so that the files are named something like [20-1.png] ... [20-168.png] if that makes it easier.

Combining the frames:

Now you need to combine the image into a spritesheet. You need to install a GIMP plugin:
https://raw.githubusercontent.com/Spydarlee/scripts/master/GIMP/create_spritesheet.py

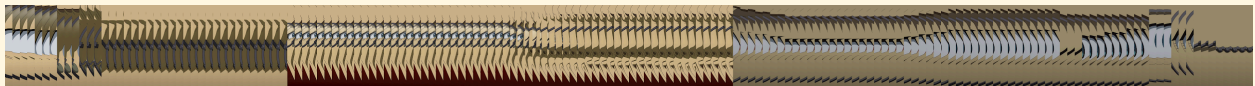
This plugin needs to be manually installed. As different versions of GIMP have different ways to install plugins, they won't be covered here. Once the plugin is installed and GIMP has been restarted, you should see "Create Spritesheet" added to Filters > Animation.

With nothing loaded in GIMP, go to File > Open as Layers, and select all the frames from the same column (e.g. [20\out1.png] to [20\out168.png]). This will take a while so you can read the next section while it loads all the frames.

(GIMP is actually a suboptimal tool to make sprite sheets in, but I don't know if a free alternative that does the same thing and allows hundreds of frames exists)

Once everything is loaded, click Create Spritesheet. Set "Output to a single row" to Yes and run it. After a few moments you should have a very wide image in a new window. If you did everything correctly, this image should have the same height as the input, and no more than 16384 pixels in width.

Save the generated strip in DDS. When prompted, choose DXT1(or BC1) compression. And just in case, save the image in PNG too. Depending on the source video, the relative size of DDS and PNG can vary greatly, so choose the one that suits your video.



This is what the generated strip should look like. There are a lot of duplicated frames in this example because Japanese anime are generally drawn at 8 frames per second.

Implementing the animation

If you haven't already, subscribe to the mod and take a look at `interface\animated_frontendmainviewbg.gfx`

Inside the file you should see 20 of the following:

```
frameAnimatedSpriteType = {  
    name = "GFX_frontend_bg_tile_01"  
    texturefile = "gfx/animated_textures/frontend_background/01.dds"  
    noOfFrames = 168  
    animation_rate_fps = 24  
    looping = yes  
    play_on_show = yes  
    pause_on_loop = 0.0  
    alwaystransparent = no  
    #effectFile = "gfx/FX/buttonstate_blendframes.lua"  
}
```

Do the same for your mod: Add one entry for each strip you have, and change the number of frames and the fps accordingly. The effectFile part is commented out, but you can experiment with it: technically it should allow a smoother animation by interpolating between frames, but the result varies greatly for different animations.

Next, look at interface\frontendmainview.gui

This file must have the exact same name in your mod too, as this is meant to override the vanilla main menu interface. The key parts are:

```
#background = {
#   name = "Background"
#   quadTextureSprite = "GFX_frontend_bg"
#   alwaystransparent = yes
#}
containerWindowType = {
    name = "frontend_background_tile_01"
    position = { x = -912 y = 0 }
    size = {
        width = 96
        height = 1080
        preserve_aspect_ratio = yes
    }
    Orientation = center
    Origo = center
    clipping = no

    background = {
        name = "Background"
        quadTextureSprite = "GFX_frontend_bg_tile_01"
        alwaystransparent = yes
    }
}
```

At the top of the file we commented out the original static background, then we added 20 additional containers. The x position is calculated from the center: since we have an even number of columns here, the two center ones would be at x=-48 and x=48 (half of width) respectively, and the rest are further offset by 96 (width) in their respective directions. This will put them in the center of the screen. If you want to place it somewhere off-center, you can also do that by setting different positions, just make sure that they have the same y-coordinate and their x-coordinates are offset by their width.

Notes:

frameAnimatedSpriteType doesn't like to be stretched, so an animated menu background will be fixed sized, and areas not covered will appear as black. For some creative ways to "hide" this shortcoming, you can keep the static background that we commented out above so that it covers the rest of the screen at higher resolutions, or you can add another static containerWindowType with a TV/monitor/projection screen texture so that it looks like the video is playing on a TV/monitor/projection screen and you can still have a static background that shows the room/planning table, etc.

Credits:

This guide is made by Miralisque of the [Youjo Senki | Redux](#) team.

Special thanks to swf541 of the [Cold War Iron Curtain](#) team for ways to work around the lack of scaling and Palpy from Palpatine's Gamble team for the concept of multi-texture background.



How it all began